STAFF REVIEW · H446 012 source models

Separate from the eight-page pupil workbook. This copy is in a public repository and is NOT access controlled. Check technical correctness before use.

Printed question mapping: Booklet Q1 = original online Q1; Booklet Q2 = original online Q4

Do Now — source answers

1. A function returns a value . A procedure-style method uses void and does not return a value.

2. do...while Its condition is checked after the body has run.

3. It compares two values for equality . = assigns; == compares.

4. Argument = 7 · parameter = number The argument is the actual value supplied; the parameter is the named input in the method definition.

I Do — source worked steps

1. Start in Main — CODE:
int score = 10; — The variable score belongs to Main() and currently stores 10.

2. The method receives a copy — CODE:
public static void ChangeByValue(int number)
{
number = 100;
} — The parameter number is a separate local variable initialised with the value 10.

3. Only number becomes 100 — CODE:
ChangeByValue(score);

inside method:
number = 100

back in Main:
score = 10 — When the method ends, its local parameter is no longer needed.

4. Reference passing changes the original — CODE:
ChangeByReference(ref score);

// method
public static void ChangeByReference(ref int number)
{
number = 100;
}

We Do — guided CHECK

1. Classify each variable

2. Track the two variables
number is a local copy, so changing it does not change x .

3. Where must ref appear?
Using ref in both places lets the method change the original variable.

4. Complete both explanations

You Do — source mark schemes

1. Local variables in two methods
CODE:
```
using System;

public class Program
{
public static void Main()
{
First();
Second();
}

public static void First()
{
int result = 10;
Console.WriteLine(result);
}

public static void Second()
{
int result = 25;
Console.WriteLine(result);
}
}
```
Why it works: each result variable is local to its own method, so the identifiers do not conflict.

2. Prove pass-by-value
CODE:
```
using System;

public class Program
{
public static void Main()
{
int score = 20;

SetToZero(score);

Console.WriteLine(score);
}

public static void SetToZero(int number)
{
```

```
number = 0;
}
}
```
Why it works: number receives a copy of the value. Changing that local copy does not change score
.

3. Change the original with ref
CODE:
```
using System;

public class Program
{
public static void Main()
{
int score = 20;

Console.WriteLine("Before: " + score);
SetToZeroByReference(ref score);
Console.WriteLine("After: " + score);
}

public static void SetToZeroByReference(ref int number)
{
number = 0;
}
}
```
Key syntax: ref appears in both the method parameter and the method call.

4. Swap two values
CODE:
```
using System;

public class Program
{
public static void Main()
{
int a = 10;
int b = 25;

Swap(ref a, ref b);

Console.WriteLine("a = " + a);
Console.WriteLine("b = " + b);
}

public static void Swap(ref int first, ref int second)
{
int temporary = first;
```

```
first = second;
second = temporary;
}
}
```

Why it works: both parameters refer to the original variables. The local temporary variable prevents one original value being lost during the swap.

Exit Ticket — source answer

1. True. For an integer passed by value in C#, the called method receives a copy.